

รายงาน Lab 7

ส่วนที่ 1: Software Design & Principles Explanation

การแบ่งหน้าที่ของคลาส (Entity, Repository, Service, Controller) ตามหลัก GRASP Patterns (เช่น Controller Pattern, High Cohesion, Low Coupling, Information Expert, Indirection)

การแบ่งหน้าที่ของคลาสดตามหลัก GRASP Patterns เป็นการแบ่งเพื่อให้แต่ละคลาสมีความรับผิดชอบที่ชัดเจน โดยประกอบด้วย Entity, Repository, Service และ Controller

1. Entity

คือ คลาสที่ใช้แทนข้อมูลหรือแนวคิดหลักของระบบ โดยมีหน้าที่เก็บ State หรือข้อมูลของ Object

Entity สอดคล้องกับหลัก Information Expert เนื่องจากเป็นการมอบความรับผิดชอบให้กับคลาสที่มีข้อมูลที่เกี่ยวข้องการทำงานนั้น

2. Repository

คือ ตัวกลางระหว่างส่วน Business Logic กับระบบจัดเก็บข้อมูล มีหน้าที่จัดการการเข้าถึงข้อมูลจากแหล่งจัดเก็บ เช่น ฐานข้อมูล (Database) ทั้งนี้ Repository ไม่ควรรับผิดชอบ Business Logic ของระบบ เช่น กฎการคำนวณหรือกฎทางธุรกิจ

Repository สอดคล้องกับหลัก High Cohesion เพราะมีหน้าที่หลักเกี่ยวกับการจัดการ Persistence โดยเฉพาะ และช่วยให้ส่วนอื่นไม่จำเป็นต้องรู้รายละเอียดของ Database

3. Service

คือ ส่วนประสานงานระหว่าง Controller, Entity และ Repository มีหน้าที่ เช่น

- รับข้อมูลจาก Controller
- ตรวจสอบหรือประมวลผลตามกฎทางธุรกิจ
- เรียกใช้ Entity หรือ Repository ที่เกี่ยวข้อง
- บันทึกหรือแก้ไขข้อมูล
- ส่งผลลัพธ์กลับไปยัง Controller

Service จึงสอดคล้องกับหลัก High Cohesion เพราะ Logic ที่เกี่ยวข้องกัน Use Case เดียวกันจะถูกรวมไว้ในส่วนที่เหมาะสม และยังสอดคล้องกับหลัก Indirection เพราะทำหน้าที่เป็นตัวกลางระหว่าง Controller กับ Repository ทำให้ Controller ไม่จำเป็นต้องติดต่อกับ Database โดยตรง

4. Controller

มีหน้าที่หลักคือ รับ Request, ประสานงาน และส่ง Response โดยรับ System Event จากภายนอก เช่น HTTP Request จากผู้ใช้ แล้วส่งต่อคำขอไปยังส่วนที่รับผิดชอบการทำงานจริง

Controller สอดคล้องกับ Controller Pattern ซึ่งกำหนดให้ Controller เป็นจุดแรกที่ได้รับคำสั่งจากภายนอกระบบ และมอบหมายงานให้ส่วนที่เหมาะสม

การประยุกต์ใช้หลักการ SOLID (SRP, OCP, LSP, ISP, DIP) ในระบบ

SOLID เป็นหลักการออกแบบซอฟต์แวร์ที่ช่วยให้ระบบมีโครงสร้างที่ดี แก้ไขและขยายได้ง่าย โดยประกอบด้วย 5 หลักการ ได้แก่ SRP, OCP, LSP, ISP และ DIP

1. SRP - Single Responsibility Principle

กล่าวว่าแต่ละคลาสควรรับผิดชอบงานของตัวเอง ไม่ควรรวมหลายหน้าที่ที่ไม่เกี่ยวข้องกันไว้ในคลาสเดียว

เช่น การแบ่งหน้าที่ระบบของคลาสต่างๆ

Entity ทำหน้าที่จัดการข้อมูลของ Object

Repository ทำหน้าที่ติดต่อ Database

Service ทำหน้าที่จัดการ Business Logic

Controller ทำหน้าที่รับและจัดการ Request

ประโยชน์ คือ ทำให้โค้ดเข้าใจง่าย ทดสอบง่าย และแก้ไขส่วนหนึ่งโดยไม่กระทบส่วนอื่นมากเกินไป

2. OCP - Open/Closed Principle

กล่าวว่าระบบควรสามารถเพิ่มความสามารถใหม่ได้โดยไม่ต้องแก้ไขโค้ดเดิมมากเกินไป เช่น การเขียน Interface แล้วใช้การ Implement เพื่อเขียนฟังก์ชันหรือสร้างคลาสใหม่

ประโยชน์ คือ ลดความเสี่ยงจากการแก้ไขโค้ดเดิม และทำให้ระบบรองรับการขยายในอนาคตได้ง่าย

3. LSP - Liskov Substitution Principle

กล่าวว่า Subclass ควรสามารถนำมาแทนที่ Superclass หรือ Interface ที่สืบทอดมาได้โดยไม่ทำให้ระบบทำงานผิดพลาดผิดปกติ กล่าวคือ สามารถใช้งานแทนตัวต้นแบบได้

ประโยชน์ คือ ทำให้การใช้ Polymorphism มีความถูกต้อง และสามารถเปลี่ยน Implementation ได้โดยไม่ทำให้ส่วนที่เรียกใช้งานพัง

4. ISP - Interface Segregation Principle

กล่าวว่า ไม่ควรบังคับให้คลาสต้อง implement Method ที่ตนเองไม่ได้ใช้งาน

ประโยชน์ คือ Interface มีขนาดเล็กและมีหน้าที่ชัดเจน ทำให้คลาสที่นำไปใช้ไม่ต้องพึ่งพาสิ่งที่ไม่จำเป็น

5. DIP - Dependency Inversion Principle

กล่าวว่า High-level module ไม่ควรขึ้นอยู่กับ Low-level module โดยตรง แต่ควรขึ้นอยู่กับ Abstraction นั่นคือลดการผูกติดระหว่างคลาส โดยใช้ Interface หรือ Abstraction เป็นตัวกลาง

ประโยชน์ คือ ลด Coupling และทำให้สามารถเปลี่ยน Implementation หรือสร้าง Mock สำหรับการทดสอบได้ง่ายขึ้น

การประยุกต์ใช้ Strategy Pattern ในการคำนวณส่วนลดราคาเกม (DiscountStrategy, NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy, DiscountContext) พร้อมประโยชน์ด้าน Open/Closed Principle (OCP)

แนวคิดของ Strategy Pattern คือการแยก Algorithm หรือวิธีการทำงานที่มีหลายรูปแบบ ออกจากคลาสหลัก แล้วกำหนดให้แต่ละรูปแบบเป็น Strategy ของตัวเอง

ในโปรเจกต์ระบบการคำนวณส่วนลดราคาเกมถูกแบ่งออกเป็นคลาสที่แตกต่างกัน โดยมี DiscountStrategy เป็น Interface ส่วน NoDiscountStrategy, StudentDiscountStrategy และ SeasonalSaleStrategy เป็น Implementation ของการคำนวณส่วนลดแต่ละรูปแบบ ขณะที่ DiscountContext ทำหน้าที่เป็นตัวกลางในการเลือก Strategy ตามประเภทส่วนลด

การออกแบบนี้จึงทำให้แต่ละคลาสมีความรับผิดชอบเฉพาะและลดการรวมเงื่อนไขการคำนวณส่วนลดไว้ในคลาสเดียว ซึ่งสอดคล้องกับ OCP - Open / Closed Principle เนื่องจากเมื่อมีความต้องการเพิ่มรูปแบบส่วนลดใหม่ สามารถสร้าง Strategy ใหม่ที่ Implement DiscountStrategy ได้ โดยไม่จำเป็นต้องแก้ไข Algorithm ของ Strategy เดิม ทำให้ระบบสามารถขยายความสามารถได้ง่ายและลดผลกระทบต่ocodeที่มีอยู่แล้ว

ทำไมต้องแยก Service Layer ออกจาก Controller และ Repository ประโยชน์ด้าน Low Coupling และ High Cohesion

การแยก Service Layer ออกมา มีเป้าหมายสำคัญคือการแยก Business Logic ออกจากส่วนที่ทำหน้าที่รับ Request และส่วนที่ทำหน้าที่ติดต่อ Database ทำให้แต่ละ Layer มีความรับผิดชอบชัดเจน และสอดคล้องกับหลัก Low Coupling และ High Cohesion

Low Coupling หมายถึง ลดการพึ่งพากันโดยตรงระหว่างคลาส โดยหากไม่มี Service อาจเป็น Controller ที่ติดต่อกับ Database โดยตรง ซึ่งทำให้เกิดการพึ่งพาสูง การมี Service จึงช่วยประสานงานและเป็นประโยชน์ต่อการแก้ไขโค้ดในภายหลังที่ไม่กระทบต่อ Controller

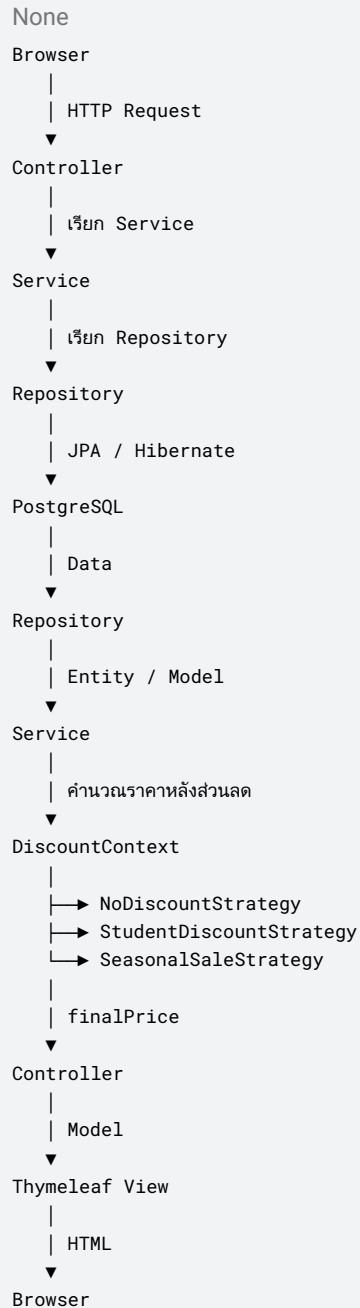
High Cohesion หมายถึง แต่ละคลาสควรมีหน้าที่ที่เกี่ยวข้องกันอย่างชัดเจน

เมื่อแบ่ง Layer ตามหน้าที่จะได้ว่า

Layer	หน้าที่หลัก
Controller	รับ Request / ส่ง Response
Service	Business Logic
Repository	Data Access
Entity	ข้อมูลและพฤติกรรมที่เกี่ยวข้องกับ Entity

ลำดับการทำงาน (Flow) เมื่อมี HTTP Request เข้ามาจาก Browser จนไปถึงการบันทึก / ดึงข้อมูลจาก PostgreSQL และคำนวณส่วนลดผ่าน Strategy Pattern

ลำดับการทำงานสามารถเขียนได้ดังนี้



อธิบายลำดับการทำงาน

เมื่อผู้ใช้เปิดระบบผ่าน Browser แล้วทำการ ดู เพิ่ม แก้ไข หรือลบข้อมูล Request จะไหลผ่านแต่ละ Layer ตามลำดับ ดังนี้

1. **Browser ส่ง HTTP Request** โดยผู้ใช้เริ่มต้นจาก Browser เช่น เข้า URL จากนั้น Browser จะส่ง HTTP Request ไปยัง Spring Boot Application แล้ว Request จะถูกส่งไปยัง Controller ที่มี Mapping ตรงกับ URL ดังกล่าว
2. **Controller รับ Request จาก Browser** และทำหน้าที่ประสานงานกับส่วนอื่น
3. **Controller ส่งต่อให้ Service** โดย Service เป็นส่วนที่จัดการ Business Logic ของระบบ ทำหน้าที่เป็นตัวกลางระหว่าง Controller กับ Repository
4. **Service เรียก Repository** โดย Repository ทำหน้าที่ติดต่อกับ Database โดยใช้ Spring Data JPA จากนั้น Repository จะแปลงการเรียกเหล่านี้ไปเป็นการทำงานกับฐานข้อมูลผ่าน JPA
5. **PostgreSQL ดึงหรือบันทึกข้อมูล**
เมื่อเป็นการดึงข้อมูล PostgreSQL จะค้นหาข้อมูลจากตารางที่เกี่ยวข้อง เช่น SELECT.... ส่วนกรณีบันทึกข้อมูล จะเกิดการทำงาน เช่น INSERT UPDATE DELETE โดยมี Entity ทำหน้าที่เป็นตัวแทนข้อมูลที่ถูก Mapping กับตารางใน PostgreSQL และข้อมูลใน PostgreSQL จะถูก Mapping กลับมาเป็น Entity/Model Object ที่ Java สามารถนำไปใช้งานต่อได้
6. **ข้อมูลถูกส่งกลับจาก PostgreSQL** โดยหลังจาก PostgreSQL ประมวลผลแล้ว ข้อมูลจะย้อนกลับตาม Layer

7. **Service คำนวณส่วนลดผ่าน Strategy Pattern** เมื่อระบบต้องแสดง ราคาหลังส่วนลด (finalPrice) ระบบจะตรวจสอบประเภทส่วนลดของเกม แล้วเลือก Strategy ที่เหมาะสมผ่าน DiscountContext เช่น

```

None
Game
|
| discountType = STUDENT
▼
DiscountContext
|
▼
StudentDiscountStrategy
|
| calculate(price)
▼
finalPrice

```

หากเป็นส่วนลดประเภทอื่นก็จะเลือก Strategy ที่แตกต่างกัน จากนั้นแต่ละ Strategy มีหน้าที่ คำนวณส่วนลดของตัวเอง

8. **Service ส่งผลลัพธ์กลับให้ Controller** เมื่อ Service ได้ข้อมูลเกมพร้อมราคาที่คำนวณแล้ว ก็จะส่งผลลัพธ์กลับไปยัง Controller จากนั้น Controller จะนำข้อมูลที่ได้ไปเตรียมสำหรับแสดงผล
9. **Controller ส่งข้อมูลไปยัง Thymeleaf** โดย Controller นำข้อมูลใส่เข้า Model จากนั้น Controller ระบุ View ที่ต้องการแสดง แล้ว Spring Boot และ Thymeleaf จะนำข้อมูลใน Model ไปแสดงใน HTML Template
10. **Thymeleaf ส่ง HTML กลับไปยัง Browser** โดย Thymeleaf จะสร้าง HTML Response แล้วส่งกลับไปยัง Browser ทำให้ผู้ใช้จึงเห็นข้อมูลบนหน้าเว็บ

ส่วนที่ 2: Code Implementation & Explanation

โครงสร้าง Code พร้อมคำอธิบาย (Entity, Repository, Strategy Package, Service, Controller) โดยอธิบายการใช้ Dependency Injection (Constructor Injection) ในทุก Layer

โครงสร้างหลักของโปรเจกเป็นดังนี้

```
None
src/main/java/com/example/demo/
|
├── model
|   └── Game.java
|
├── repository
|   └── GameRepository.java
|
├── strategy
|   ├── DiscountStrategy.java
|   ├── DiscountContext.java
|   ├── NoDiscountStrategy.java
|   ├── StudentDiscountStrategy.java
|   └── SeasonalSaleStrategy.java
|
├── service
|   └── GameService.java
|
└── controller
    └── GameController.java
```

การแบ่งหน้าที่ของคลาส

1. Entity / Model

คลาส Game (Game.java) ทำหน้าที่เป็นข้อมูลหลักของระบบเกมและเป็น JPA Entity ที่เชื่อมกับตารางในฐานข้อมูล PostgreSQL

โครงสร้างหลักเป็นดังนี้

```
@Entity
@Table(name = "lab7demo")
public class Game {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String genre;
    private String platform;
    private Double rating;
    private LocalDate releaseDate;
    private Double price;
    private String discountType;
```

โดย @Entity บอกว่า Game เป็น JPA Entity

ส่วน @Table(name = "lab7demo") เป็นการกำหนดให้ Entity นี้ Mapping กับตาราง lab7demo

Constructor สำหรับสร้าง Object

```
// Constructor
public Game() {
}

public Game(String title, String genre, String platform, Double rating, LocalDate releaseDate, Double price,
            String discountType) {
    this.title = title;
    this.genre = genre;
    this.platform = platform;
    this.rating = rating;
    this.releaseDate = releaseDate;
    this.price = price;
    this.discountType = discountType;
}
```

Dependency Injection ใน Entity

Entity ในโปรเจกต์นี้ไม่ได้ใช้ Constructor Injection เนื่องจาก Constructor เหล่านี้ใช้สำหรับสร้างและกำหนดข้อมูลของ Entity ไม่ได้ใช้รับ Dependency จาก Spring โดยเฉพาะ Default Constructor จำเป็นสำหรับ JPA ในการสร้าง Entity จากข้อมูลใน Database

2. Repository

GameRepository (GameRepository.java) มีหน้าที่จัดการ Data Access เช่น

- findAll()
- findById()
- save()
- existsById()
- deleteById()

โดย Method เหล่านี้ได้รับมาจาก JpaRepository

Dependency Injection ใน Repository

Repository ในโปรเจกต์นี้ไม่ได้ใช้ Constructor Injection เนื่องจาก GameRepository เป็น interface และไม่ได้สร้าง Object ด้วย Constructor ของเราเอง Spring Data JPA จะสร้าง Implementation ของ Repository ให้โดยอัตโนมัติ

3. Strategy Package

3.1. DiscountStrategy.java เป็น Interface กลาง ที่กำหนด Contract ว่า Strategy ทุกประเภทต้องมี calculate(double price)

```
public interface DiscountStrategy {

    double calculate(double price);

}
```

3.2. NoDiscountStrategy.java ทำหน้าที่คำนวณราคากรณีไม่มีส่วนลด

```
public class NoDiscountStrategy implements DiscountStrategy {

    @Override
    public double calculate(double price){
        return price;
    }

}
```

3.3. StudentDiscountStrategy.java ทำหน้าที่คำนวณราคากรณีมีส่วนลด 10% จากราคาเดิม

```
public class StudentDiscountStrategy implements DiscountStrategy {

    @Override
    public double calculate(double price) {
        return price * 0.9;
    }

}
```

3.4. SeasonalSaleStrategy.java ทำหน้าที่คำนวณราคากรณีมีส่วนลด 20% จากราคาเดิม

```
public class SeasonalSaleStrategy implements DiscountStrategy {
    @Override
    public double calculate(double price) {
        return price * 0.8;
    }
}
```

3.5. DiscountContext.java ทำหน้าที่เลือก Strategy ตามประเภทส่วนลด ดังโค้ดด้านล่าง

```
public class DiscountContext {
    public static DiscountStrategy getStrategy(String type) {
        if (type == null) {
            return new NoDiscountStrategy();
        }
        switch (type.toUpperCase()) {
            case "STUDENT":
                return new StudentDiscountStrategy();
            case "SEASONAL":
                return new SeasonalSaleStrategy();
            default:
                return new NoDiscountStrategy();
        }
    }
}
```

Dependency Injection ใน Strategy

Strategy ทั้ง 3 คลาส ในโปรเจกต์นี้ไม่ได้ใช้ Dependency Injection เพราะไม่มี Dependency อื่นที่ต้องส่งเข้ามาทาง Constructor

เช่น calculate ใน NoDiscountStrategy.java

```
@Override
public double calculate(double price){
    return price;
}
```

ที่ใช้เพียง price ที่ส่งเข้ามาเพื่อคำนวณเท่านั้น

Dependency Injection ใน DiscountContext

DiscountContext ในโปรเจกต์นี้ไม่ได้ใช้ Constructor Injection เพราะ DiscountContext ใช้ Static Method และสร้าง Strategy ด้วย new StudentDiscountStrategy() โดยตรง ซึ่งทำหน้าที่เป็นตัวเลือก Strategy และสร้าง Strategy Implementation ตามประเภทส่วนลด

4. Service

GameService (GameService.java) ทำหน้าที่จัดการ Business Logic และเป็นตัวกลางระหว่าง GameController กับ GameRepository โดยมีหน้าที่ดังนี้

4.1. ค้นหาข้อมูลเกมทั้งหมดที่อยู่ในฐานข้อมูล ด้วยคำสั่ง getAllGames()

```
// Read All
public List<Game> getAllGames() {
    return repository.findAll();
}
```

4.2. ค้นหาข้อมูลเกมตามไอดี ด้วยคำสั่ง getGameById(Long id)

```
// Read One
public Game getGameById(Long id) {
    return repository.findById(id).orElse(other: null);
}
```

4.3. เพิ่มข้อมูลเกมในฐานข้อมูล ด้วยคำสั่ง addGame(Game game)

```
// Add
public Game addGame(Game game) {
    return repository.save(game);
}
```

4.4. อัปเดตข้อมูลเกมในฐานข้อมูล ด้วยคำสั่ง updateGame(Long id, Game game)

```
// Update
public Game updateGame(Long id, Game game) {

    Game oldGame = repository.findById(id).orElse(other: null);

    if (oldGame != null) {
        oldGame.setTitle(game.getTitle());
        oldGame.setGenre(game.getGenre());
        oldGame.setPlatform(game.getPlatform());
        oldGame.setRating(game.getRating());
        oldGame.setReleaseDate(game.getReleaseDate());
        oldGame.setPrice(game.getPrice());
        oldGame.setDiscountType(game.getDiscountType());

        return repository.save(oldGame);
    }

    return null;
}
```

4.5. ลบข้อมูลเกมในฐานข้อมูล ด้วยคำสั่ง deleteGame(Long id)

```
// Delete
public boolean deleteGame(Long id) {

    if (repository.existsById(id)) {
        repository.deleteById(id);
        return true;
    }
}
```

4.6. รับ GameRepository ผ่าน Constructor Injection เพื่อให้ Service สามารถเรียกใช้ Repository ในการเข้าถึงฐานข้อมูล โดยไม่ต้องสร้าง Repository ขึ้นมาเอง

```
@Service
public class GameService {

    private final GameRepository repository;
```

Dependency Injection ใน Service

GameService ใช้ Constructor Injection โดยมีการประกาศ Dependency คือ GameRepository

```
@Service
public class GameService {

    private final GameRepository repository;
```

และรับ Dependency ผ่าน Constructor

```
public GameService(GameRepository repository) {
    this.repository = repository;
}
```

กล่าวคือ Spring จะเป็นผู้ส่ง GameRepository เข้ามาให้ GameService Service จึงไม่ต้องสร้าง Repository เอง

5. Controller

GameController (GameController.java) ทำหน้าที่รับ HTTP Request จาก Browser และเรียกใช้งาน GameService เช่น

```
@GetMapping
public String listGames(Model model) {
    model.addAttribute("games", gameService.getAllGames());
    return "games/list";
}
```

จากโค้ดจะเห็นว่า Controller ไม่เข้าถึง Repository โดยตรง แต่เรียกผ่าน Service

Dependency Injection ใน Controller

GameController ใช้ Constructor Injection โดยมีการประกาศ Dependency คือ GameService

```
@Controller
@RequestMapping("/games")
public class GameController {

    private final GameService gameService;
```

และรับ Dependency ผ่าน Constructor

```
// Constructor Injection
public GameController(GameService gameService) {
    this.gameService = gameService;
}
```

จากนั้น Spring จะสร้าง GameService และ Inject เข้ามาใน GameController

ส่วนที่ 3: Web Application & Database Screenshots

การเพิ่มเกมใหม่ หน้าจอเพิ่มเกมใหม่ (Create)

← กลับไปรายการเกม

+ เพิ่มเกมใหม่
กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

ชื่อเกม
The Champignon (673380299-1 SEC 2)

แนวเกม (Genre)
Adventure, RPG, 2D-Platform

แพลตฟอร์ม (Platform)
PC

☆ คะแนน (Rating)
9

💰 ราคาปกติ (บาท)
1000

% ส่วนลด (Strategy)
ส่วนลดนักศึกษา (10%)

📅 วันวางจำหน่าย (Release Date)
12/08/2026

บันทึก **ยกเลิก**

หน้าจอแสดงรายการเกมทั้งหมด (Read)

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด
จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

+ เพิ่มเกมใหม่

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Resident Evil 4 Remake	Action, Survival Horror, Third-Person Shooter	PC, PS5	10.0	05/08/2026	NONE	1200.00	 
2	The Champignon (673380299-1 SEC 2)	Adventure, RPG, 2D-Platform	PC	9.0	12/08/2026	STUDENT	900.00 1000.00	 
3	Planet Zoo (673380299-1 SEC 2)	Simulation, Strategy, Management, Casual, Sandbox	PC	9.8	12/08/2026	SEASONAL	1192.00 1490.00	 

📄 ทั้งหมด 3 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

หน้าจอแก้ไขเกม (Update)

← กลับไปรายการเกม

แก้ไขข้อมูลเกม

แก้ไขข้อมูลเกม: The Champignon (673380299-1 SEC 2)

ชื่อเกม

The Champignon (673380299-1 SEC 2)

แนวเกม (Genre)

Adventure, RPG, 2D-Platform

แพลตฟอร์ม (Platform)

PC

คะแนน (Rating)

9.0

ราคาปกติ (บาท)

1000.0

% ส่วนลด (Strategy)

ส่วนลดนักศึกษา (10%)

วันวางจำหน่าย (Release Date)

วว/ดด/ปปปป

อัปเดต **ยกเลิก**

หน้าจอยืนยันลบ

← กลับไปรายการเกม

ยืนยันการลบ

คุณแน่ใจหรือไม่?

การดำเนินการนี้จะลบข้อมูลเกมออกจากระบบอย่างถาวร

ชื่อเกม

Planet Zoo (673380299-1 SEC 2)

แนวเกม

Simulation, Strategy, Management, Casual, Sandbox

แพลตฟอร์ม

PC

คะแนน

★ 9.8

ราคา

1490.00 บาท

ยืนยันลบ **← ยกเลิก**

ผลลัพธ์หลังลบ (Delete)

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

เพิ่มเกมใหม่

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Resident Evil 4 Remake	Action, Survival Horror, Third-Person Shooter	PC, PS5	★ 10.0	05/08/2026	NONE	1200.00	
2	The Champignon (673380299-1 SEC 2)	Adventure, RPG, 2D-Platform	PC	★ 9.0	12/08/2026	STUDENT	900.00 1000.00	

ทั้งหมด 2 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

หน้าจอ Database

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (2)
 - lab7demo
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - postgres
 - Login/Group Roles
 - Tablespaces

lab7demo/postgres@PostgreSQL 18*

Query

```
SELECT * FROM lab7demo;
```

Query History

Scratch Pad

Data Output

Showing rows: 1 to 2 of 1

	discount_type	genre	platform	price	rating	release_date	title
1	1 NONE	Action, Survival Horror, Third-Person Shoo...	PC, PS5	1200	10	2026-08-05	Resident Evil 4 Remake
2	4 STUDENT	Adventure, RPG, 2D-Platform	PC	1000	9	2026-08-12	The Champignon (673380299-1 SEC...